

Newtrinos.jl: A Julia Package for Global Analysis of Neutrino Data

Philipp Eller¹ and David Schultheiß¹

¹ Technical University of Munich, Germany

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Matthew Feickert](#)

Reviewers:

- [@rafaelab](#)
- [@szabo137](#)

Submitted: 28 October 2025

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Newtrinos.jl is an open-source Julia package for performing global analyses of neutrino data. It provides a modular, three-layer architecture that separates physics models, experiment descriptions, and statistical inference into independent modules. This allows researchers to freely combine experiments and test them against a variety of theoretical models. New experiments and physics models can be added without modifying core code.

Full statistical forward models including all relevant systematic uncertainties are implemented for each experimental dataset, defining both the likelihood and the data-generating process and enabling modern statistical inference workflows.

The framework is composable, making it straightforward to construct joint likelihoods over multiple datasets. Physics and nuisance parameters can be merged, correlated, or decorrelated across experiments to ensure consistency in the joint fit.

The package supports Frequentist and Bayesian inference and is parallelizable across CPU threads or distributed workers. Written entirely in Julia, all models are automatically differentiable, enabling exact gradient computation.

Statement of Need

As neutrino physics transitions into an era of high-precision measurements, global fits have become essential for extracting neutrino properties and probing physics beyond the Standard Model ([Capozzi et al., 2025](#); [Esteban et al., 2024](#); [Salas et al., 2021](#)). By combining disparate datasets, they break parameter degeneracies that limit single-experiment analyses. Consequently, they are poised to address urgent open questions such as the neutrino mass ordering and CP violation in the lepton sector. The first joint fit between T2K and NOvA demonstrated that combining distinct experimental signatures is crucial to resolving such degeneracies and tensions ([Abubakar & others, 2025](#)).

Performing a global neutrino fit requires assembling diverse, experiment-specific detector models, neutrino flux models, matter density profiles, interaction cross sections, et cetera, into a coherent inference pipeline, while rigorously accounting for large numbers of correlated systematic uncertainties. Varying availability of public data and documentation further limits dataset compatibility, making global fits a substantial undertaking that can render certain analyses infeasible ([Capozzi et al., 2025](#)). Existing frameworks that have enabled successful studies are typically proprietary and closed-source, making results difficult to verify and the software impossible to reuse or adapt.

A further challenge is the computational burden of high-dimensional parameter spaces with correlated systematics. As the number of experiments grows, traditional statistical methods become increasingly intractable. Automatic differentiation and gradient-based inference offer

promising paths forward. However, existing frameworks (see Related Work) were developed before these techniques became widely adopted and therefore rely primarily on derivative-free optimization or finite-difference gradient estimates such as those used by MINUIT (James & Roos, 1975).

Newtrinos.jl addresses these limitations with a fully open-source, extensible framework built within the high-performance Julia ecosystem.

Key Features

Key features include:

- **Out-of-the-box usability:** The package includes experimental data, configuration files, and plotting tools, making it straightforward to reproduce results, verify correctness, and conduct custom analyses on existing models and experiments.
- **Modular architecture:** Experiments, physics models, and analysis methods are implemented as independent components with clearly separated concerns. The physics layer provides theory predictions and physical parameters; the experiment layer configures individual experiments with their parameters, priors, and forward models; the analysis layer provides inference methods for sampling, profiling, or scanning. Each layer treats the others as black boxes, enabling flexible composition. Layers communicate through two well-defined interfaces: `NamedTuples` of parameters and priors adhering to the `Distributions.jl` standard, and callable functions stored in structs. Theory extensions can be added via Julia's multiple dispatch without modifying existing code, and parameters flow through the pipeline automatically.
- **Full statistical models:** The package constructs full statistical forward models for each experiment, encoding the complete mapping from physical parameters to expected observations. Systematic uncertainties enter as nuisance parameters rather than being absorbed into simplified χ^2 approximations, preserving the full statistical information. This enables profiling or marginalisation of joint likelihoods, Bayesian posterior estimation, and the generation of pseudo-data and prior and posterior predictive checks.
- **Consistent parameter handling:** All physics and nuisance parameters across configured experiments are collected into a single, flat `NamedTuple` via `get_params` and `get_priors` with `safe_merge`. Wrapper functions support arbitrary parameter renaming and (de)correlation between experiments. Prior distributions, including correlated priors via covariance matrices, are managed in the same unified structure and compose naturally into a joint prior. Individual parameters can be overridden or conditioned at analysis time without modifying the underlying modules, keeping configurations transparent and reproducible.
- **Automatic differentiation:** The complete forward model chain — from oscillation probability calculation to likelihood evaluation — is compatible with *ForwardDiff.jl* out of the box, with *Mooncake.jl* also successfully tested. Exact, efficient gradients enable advanced optimisation and inference methods such as those provided by (Schulz et al., 2021).
- **Scalability:** The package is built for large-scale inference tasks with tens of experiments and hundreds of parameters. Julia's just-in-time compilation delivers high performance without sacrificing expressiveness. Profile likelihood scans parallelise transparently across CPU threads or distributed workers via `Distributed.jl`, and forward-mode automatic differentiation keeps gradient-based inference tractable as the parameter space grows.

Workflow Overview

Newtrinos.jl is designed as a modular pipeline where components communicate strictly through functional interfaces: physics modules expose callables used inside experiment forward models, and experiments expose forward model callables consumed by the analysis layer, without either inspecting the other's internals. A typical analysis proceeds as follows:

1. **Configure physics:** Select and instantiate theoretical models — for example, a three-flavour oscillation model with non-standard matter interactions, an Earth density model, an atmospheric flux model, and a cross-section model. Any model can be swapped independently, e.g. replacing a three-flavour model with a BSM model including sterile neutrinos.
2. **Configure experiments:** Select one or more experiment modules, each encapsulating its detector response, systematic uncertainties, and observed data. Instantiate them via the `configure` method with the chosen physics model. Experiments are independent and can be freely combined, typically sharing a common physics model.
3. **Build the joint likelihood:** Pass the collection of configured experiments to `generate_likelihood`, which composes their individual forward models and likelihood functions into a single joint likelihood.
4. **Collect parameters and priors:** Use `get_params` and `get_priors` to collect and merge all physics and nuisance parameters across experiments into a unified `NamedTuple`. Parameters and priors can be modified as needed, e.g. for likelihood conditioning via `Accessors.jl`.
5. **Run inference:** Choose an analysis method from the provided tools. For example, run a profile likelihood scan by calling `profile` on the joint likelihood over a chosen parameter grid to obtain a `NewtrinosResult` containing the grid coordinates, likelihood values, optimised nuisance parameters at each point, and run metadata.
6. **Visualize and export results:** Plot confidence contours or best-fit data/MC comparisons using the built-in plotting utilities based on `Makie.jl`, and save results to disk.

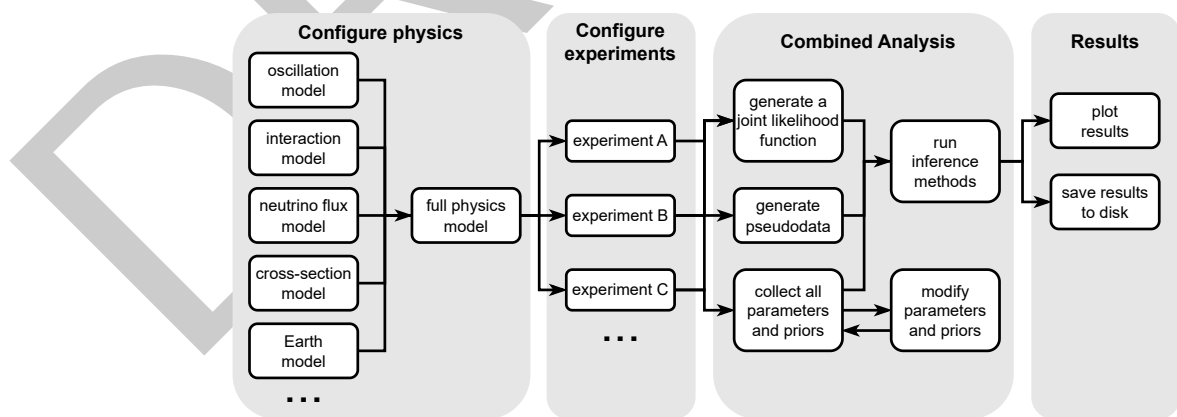


Figure 1: Typical workflow for global analyses of neutrino data with *Newtrinos.jl*. The layers communicate through two well-defined interfaces: `NamedTuples` of parameters and priors, and callable functions stored in structs.

Availability

Newtrinos.jl is open-source and freely available under the MIT License at <https://github.com/philippeller/Newtrinos.jl>

Related Work

Several existing software projects address related but distinct use cases:

- **GLOBES** (Huber et al., 2005, 2007): Simulates long-baseline experiments but does not support full global fits.
- **GAMBIT** (Athron & others, 2017): A general-purpose global fitting framework with some support for neutrino data, but not tailored for neutrino physics.
- **PEANUTS** (Gonzalo & Lucente, 2024): Focused on solar neutrino modelling.
- **PhyLiNO** (Hellwig et al., 2025): A high-performance framework for reactor neutrino data.
- **PISA** (Aartsen & others, 2020): Designed for atmospheric neutrino analyses.

Newtrinos.jl complements these efforts by focusing on the neutrino sector, offering a simple, extensible, and efficient design. It is currently the only framework in this domain supporting automatic differentiation. The software has been used in (Ettengruber et al., 2024), (Kozynets et al., 2025), (Eller et al., 2025), and (Eller, 2026).

Acknowledgements

This work was supported by Germany's Federal Ministry of Research, Technology and Space (BMFTR) within the ErUM-Data programme under grant FKZ 05D25PC1 (DEMOS consortium), and partially by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy – EXC-2094/2 – 390783311, the SFB 1258–283604770, and NFDI 39/1.

Aartsen, M. G., & others. (2020). Computational techniques for the analysis of small signals in high-statistics neutrino oscillation experiments. *Nucl. Instrum. Meth. A*, 977, 164332. <https://doi.org/10.1016/j.nima.2020.164332>

Abubakar, S., & others. (2025). Joint neutrino oscillation analysis from the T2K and NOvA experiments. *Nature*, 646, 818–824. <https://doi.org/10.1038/s41586-025-09599-3>

Athron, P., & others. (2017). GAMBIT: The Global and Modular Beyond-the-Standard-Model Inference Tool. *Eur. Phys. J. C*, 77(11), 784. <https://doi.org/10.1140/epjc/s10052-017-5321-8>

Capozzi, F., Giarè, W., Lisi, E., Marrone, A., Melchiorri, A., & Palazzo, A. (2025). Neutrino masses and mixing: Entering the era of subpercent precision. *Phys. Rev. D*, 111(9), 093006. <https://doi.org/10.1103/PhysRevD.111.093006>

Eller, P. (2026). *Atmospheric Neutrino Oscillations: the Full Picture*. <https://arxiv.org/abs/2606.09714>

Eller, P., Ettengruber, M., & Zander, A. (2025). Neutrino data analysis of extra-dimensional theories with massive bulk fields. *Phys. Rev. D*, 112(5), 055009. <https://doi.org/10.1103/11m-96vy>

Esteban, I., Gonzalez-Garcia, M. C., Maltoni, M., Martinez-Soler, I., Pinheiro, J. P., & Schwetz, T. (2024). NuFit-6.0: updated global analysis of three-flavor neutrino oscillations. *JHEP*, 12, 216. [https://doi.org/10.1007/JHEP12\(2024\)216](https://doi.org/10.1007/JHEP12(2024)216)

Ettengruber, M., Zander, A., & Eller, P. (2024). Testing the number of neutrino species with

- 155 a global fit of neutrino data. *Phys. Rev. D*, 109(9), 095016. [https://doi.org/10.1103/](https://doi.org/10.1103/PhysRevD.109.095016)
156 [PhysRevD.109.095016](https://doi.org/10.1103/PhysRevD.109.095016)
- 157 Gonzalo, T. E., & Lucente, M. (2024). PEANUTS: a software for the automatic computation
158 of solar neutrino flux and its propagation within Earth. *Eur. Phys. J. C*, 84(2), 119.
159 <https://doi.org/10.1140/epjc/s10052-024-12423-3>
- 160 Hellwig, D., Schoppmann, S., Soldin, P., Stahl, A., & Wiebusch, C. (2025). PhyLiNO: a
161 forward-folding likelihood-fit framework for neutrino oscillation physics. *Comput. Softw.*
162 *Big Sci.*, 9(1), 11. <https://doi.org/10.1007/s41781-025-00142-7>
- 163 Huber, P., Kopp, J., Lindner, M., Rolinec, M., & Winter, W. (2007). New features in the
164 simulation of neutrino oscillation experiments with GLoBES 3.0: General Long Baseline
165 Experiment Simulator. *Comput. Phys. Commun.*, 177, 432–438. [https://doi.org/10.1016/](https://doi.org/10.1016/j.cpc.2007.05.004)
166 [j.cpc.2007.05.004](https://doi.org/10.1016/j.cpc.2007.05.004)
- 167 Huber, P., Lindner, M., & Winter, W. (2005). Simulation of long-baseline neutrino oscillation
168 experiments with GLoBES (General Long Baseline Experiment Simulator). *Comput. Phys.*
169 *Commun.*, 167, 195. <https://doi.org/10.1016/j.cpc.2005.01.003>
- 170 James, F., & Roos, M. (1975). Minuit: A System for Function Minimization and Analysis
171 of the Parameter Errors and Correlations. *Comput. Phys. Commun.*, 10, 343–367.
172 [https://doi.org/10.1016/0010-4655\(75\)90039-9](https://doi.org/10.1016/0010-4655(75)90039-9)
- 173 Kozynets, T., Eller, P., Zander, A., Ettengruber, M., & Koskinen, D. J. (2025). Constraints
174 on non-unitary neutrino mixing in light of atmospheric and reactor neutrino data. *JHEP*,
175 05, 130. [https://doi.org/10.1007/JHEP05\(2025\)130](https://doi.org/10.1007/JHEP05(2025)130)
- 176 Salas, P. F. de, Forero, D. V., Gariazzo, S., Martínez-Miravé, P., Mena, O., Ternes, C. A.,
177 Tórtola, M., & Valle, J. W. F. (2021). 2020 global reassessment of the neutrino oscillation
178 picture. *JHEP*, 02, 071. [https://doi.org/10.1007/JHEP02\(2021\)071](https://doi.org/10.1007/JHEP02(2021)071)
- 179 Schulz, O., Beaujean, F., Caldwell, A., Grunwald, C., Hafych, V., Kröninger, K., Cagnina, S.
180 L., Röhrig, L., & Shtembari, L. (2021). BAT.jl: A julia-based tool for bayesian inference.
181 *SN Computer Science*, 2(3), 210. <https://doi.org/10.1007/s42979-021-00626-4>